C PROGRAMMING · CORE CONCEPTS

STORAGE CLASSES

IN C PROGRAMMING

A Complete Exam-Oriented Study Guide

auto • register • static (local & global) • extern • scope rules

For Competitive Examinations · University Semester Exams · Teacher Recruitment Tests (TET/TGT/PGT/NET/GATE)

TABLE OF CONTENTS

1. Introduction to Storage Classes
2. Automatic Storage Class (auto)
3. Register Storage Class (register)
4. Static Storage Class (static) — Local & Global
5. External Storage Class (extern)
6. Scope Rules in C
7. Comparison of Storage Classes
8. Programming Examples
9. Quick Revision Section

CHAPTER 1

Introduction to Storage Classes

1.1 Definition

A **storage class** in C determines several important properties of a variable, including its storage location, scope (visibility), lifetime (storage duration), and default initialization. Specifically, it defines:

- **Storage location** – where the variable is typically stored in memory (such as a CPU register, stack memory, or the data segment).
- **Scope** – the region of the program in which the variable can be accessed.
- **Lifetime (storage duration)** – how long the variable exists during program execution.
- **Default initialization** – the initial value assigned to the variable if no explicit initialization is provided.

C provides four storage class specifiers: **`auto`**, **`register`**, **`static`**, and **`extern`**.

1.2 Why Storage Classes Are Needed

- To control the **visibility** of a variable — restrict or extend where it can be used.
- To control the **lifetime** of a variable — whether it is destroyed after a block/function ends or persists.
- To optimize **program speed** by placing frequently used variables in CPU registers.
- To enable **sharing of variables across multiple files** in large, multi-file projects.
- To manage **memory efficiently** by choosing stack, register, or static storage as appropriate.

1.3 Diagram: Memory Organization in C

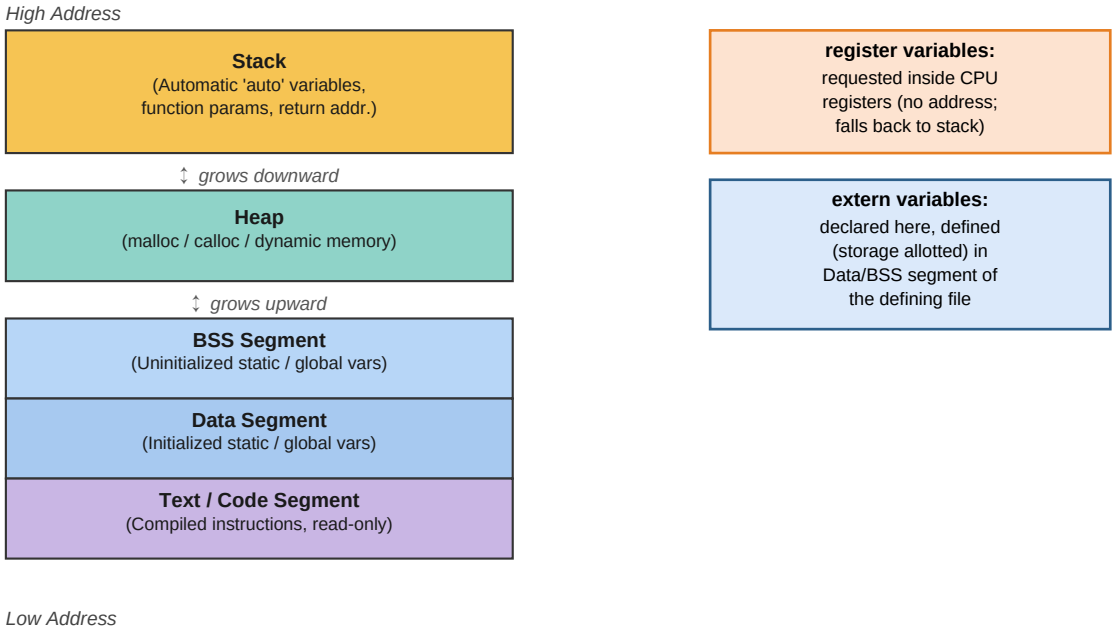

Fig 1.1 — A running C program's memory is divided into Text (code), Data/BSS (global & static variables), Heap (dynamic memory), and Stack (automatic variables & function call frames).

★ EXAM TIP: Exam Favourite

Remember the mnemonic **"A-R-S-E"** for the four storage classes: **A**uto, **R**egister, **S**tatic, **E**xtern. Most objective questions test default value, scope, and lifetime of each.

CHAPTER 2

Automatic Storage Class (auto)

2.1 Definition

The **`auto`** storage class is the **default** storage class for all local (block-scope) variables declared inside a function or block. Every local variable that is not explicitly declared as **`static`**, **`register`**, or **`extern`** is automatically an **`auto`** variable.

2.2 Syntax

Syntax

```
auto data_type variable_name;
auto data_type variable_name = value;

/* the keyword 'auto' is almost always omitted */
data_type variable_name; // implicitly auto
```

2.3 Key Properties

Property	Description
Default behaviour	Applied automatically to every local variable if no other storage class is specified.
Scope	Block scope — visible only within the { } block in which it is declared.
Lifetime	Exists only while the block/function is executing; destroyed on exit from the block.
Storage location	RAM — specifically the **stack** segment (as part of the function's stack frame).
Default value	Garbage / indeterminate (NOT auto-initialized to zero).
Re-entrant?	Yes — a fresh copy is created on every call; recursion works correctly.

2.4 Example with Output

auto_demo.c

```
#include <stdio.h>

void counter(void) {
auto int count = 0; /* re-initialized to 0 every call */
count = count + 1;
printf("count = %d\n", count);
}

int main(void) {
counter();
counter();
counter();
return 0;
```

```
}
```

Output

```
count = 1
count = 1
count = 1
```

Because **`count`** is **`auto`**, it is created afresh (and re-initialized) on every call to `counter()`; it does not retain its value between calls.

■ COMMON MISTAKE: Uninitialized auto Variables

If an **`auto`** variable is used before assigning it a value, it contains **garbage** (whatever was previously in that stack memory). Always initialize local variables explicitly.

2.5 Advantages and Limitations

Advantages	Limitations
Automatic memory management — no manual cleanup needed.	Value is lost once the block/function ends.
Efficient use of memory (stack is reused across calls).	Cannot be accessed outside its own block/function.
Supports recursion safely (separate copy per call).	Contains garbage value if not explicitly initialized.

CHAPTER 3

Register Storage Class (register)

3.1 Definition

The **`register`** storage class requests the compiler to store a variable in a **CPU register** instead of RAM, so that it can be accessed much faster. It is typically used for variables that are accessed very frequently, such as loop counters.

3.2 Syntax

Syntax

```
register data_type variable_name;
register data_type variable_name = value;

register int i;
for (i = 0; i < 1000; i++) { /* fast access */ }
```

3.3 Key Properties

Property	Description
Purpose	Hint to the compiler for faster access by storing the variable in a CPU register.
Scope	Block scope — same as auto (visible only inside its own block/function).
Lifetime	Same as auto — exists only during execution of the block/function.
Storage location	CPU register, if available; otherwise the compiler silently falls back to the stack (RAM), same as auto.
Default value	Garbage / indeterminate, like auto.
Address-of (&)	The unary **&** operator CANNOT be applied — registers have no memory address.

3.4 Restrictions and Limitations

- Only a **limited number of CPU registers** exist, so only a few variables can actually be placed in registers.
- The compiler may **ignore** the register request (it is only a suggestion, not a command) and treat the variable as auto.
- The **&** (address-of) operator cannot be used on a register variable — this is a compile-time error.
- Modern optimizing compilers usually decide register allocation themselves, making explicit 'register' largely obsolete in modern C (register keyword is a no-op suggestion in most compilers, but still valid syntax).
- Since C++17, 'register' keyword usage is deprecated; in C it remains valid but rarely necessary.

3.5 Example with Output

register_demo.c

```
#include <stdio.h>

int main(void) {
register int i; /* hint: keep i in a CPU register */
```

```
long sum = 0;

for (i = 1; i <= 5; i++) {
sum = sum + i;
}
printf("Sum = %ld\n", sum);
/* printf("%p", &i); ERROR: cannot take address of register var */
return 0;
}
```

Output

```
Sum = 15
```

✓ NOTE: Compiler Behaviour

In modern compilers (GCC, Clang), the **`register`** keyword rarely changes actual code generation — optimizers choose register allocation automatically. However, it is still **frequently asked in exams** for its rules (especially the address-of restriction).

CHAPTER 4

Static Storage Class (static)

Definition: *The* `static` *storage class makes a variable* ***retain*** *(keep / hold onto) its value for the entire program and, depending on where it is declared, is classified as either a* ***local static variable*** *(inside a function/block) or a* ***global static variable*** *(outside all functions).*

4(a). Local Static Variables

4.1 Definition

A variable declared with **`static` inside a function/block** is called a **local static variable**. Unlike auto variables, it is initialized **only once** and **retains its value between successive function calls**.

4.2 Syntax

Syntax

```
static data_type variable_name;
static data_type variable_name = value; // initialized only once

static int count = 0;
```

4.3 Key Properties

Property	Description
Scope	Block scope — visible only inside the function/block where declared (same as auto).
Lifetime	Entire program execution — memory is allocated once and persists till the program ends.
Storage location	Data segment (if initialized) or BSS segment (if uninitialized) — NOT the stack.
Default value	0 (zero) if not explicitly (directly, clearly, on purpose) initialized — unlike auto/register.
Initialization	Happens only ONCE, the first time control reaches the declaration; ignored on later calls.
Persistence	Retains its last updated value across multiple function calls (the value persists — continues to exist / stays around without disappearing).

4.4 Example with Output

static_local_demo.c

```
#include <stdio.h>

void counter(void) {
static int count = 0; /* initialized ONLY once */
count = count + 1;
printf("count = %d\n", count);
}

int main(void) {
```

```
counter();
counter();
counter();
return 0;
}
```

Output

```
count = 1
count = 2
count = 3
```

Compare this with the identical **`auto`** example in Chapter 2 — there, output was 1,1,1. Here, because `count` is **`static`**, its value **persists** across calls, giving 1, 2, 3.

4(b). Global Static Variables

4.5 Definition

A variable declared with **`static` outside all functions** (at file level) is a **global static variable**. It behaves like an ordinary global variable in scope within the file, but its **visibility is restricted to that file only** — it cannot be accessed from other files even with **`extern`**.

4.6 Visibility Rules, Lifetime & Usage

Property	Description
Scope	File scope — accessible anywhere within the same file, from point of declaration to end of file.
Visibility to other files	NOT accessible from other files, even using extern (this is its key purpose — encapsulation).
Lifetime	Entire program execution.
Storage location	Data segment / BSS segment.
Default value	0 (zero) if not explicitly initialized.
Typical usage	Hiding implementation details / internal counters in multi-file (modular) programs to avoid naming clashes.

4.7 Example with Output

static_global_demo.c

```
#include <stdio.h>

static int moduleCalls = 0; /* file-scope static: hidden from other files */

void track(void) {
moduleCalls++;
printf("This file's function called %d time(s)\n", moduleCalls);
}

int main(void) {
track();
track();
```

```
return 0;
}
```

Output

```
This file's function called 1 time(s)
This file's function called 2 time(s)
```

★ EXAM TIP: auto vs static — One-Line Rule

auto = new value every call (lost on exit). **static (local)** = value survives between calls (initialized once). **static (global)** = global variable, but confined/private to its own file.

CHAPTER 5

External Storage Class (extern)

5.1 Definition

The **`extern`** storage class is used to **declare** a global variable or function that is **defined in another file** (or later in the same file), allowing that variable to be **shared across multiple source files** in a multi-file C program.

5.2 Syntax

Syntax

```
extern data_type variable_name; /* DECLARATION — no memory allocated, no initializer */

/* Actual DEFINITION (allocates memory) appears in exactly ONE file: */
data_type variable_name = value;
```

5.3 Key Properties

Property	Description
Scope	Global — can span multiple files (the widest scope among all storage classes).
Lifetime	Entire program execution.
Storage location	Data segment / BSS segment (memory allocated only at the point of definition).
Default value	0 (zero) if not explicitly initialized.
Declaration vs Definition	'extern' only DECLARES (announces existence); memory is allocated by exactly one DEFINITION elsewhere.
Cannot be initialized	An 'extern' declaration cannot carry an initializer, e.g. extern int x = 5; is invalid usage as a pure declaration.

5.4 Working Mechanism — Sharing Variables Between Files

In a multi-file project, a variable is **defined** (memory allocated) in exactly one .c file, and **declared** using **`extern`** in every other file that needs to use it. At link time, the linker resolves all extern references to the single actual definition.

5.5 Multi-File Example with Output

file1.c (definition)

```
#include <stdio.h>

int shared_value = 100; /* DEFINITION: memory allocated here */

void display(void); /* function prototype */

int main(void) {
printf("In file1: shared_value = %d\n", shared_value);
shared_value += 50;
display();
```

```
return 0;
}
```

file2.c (uses the same variable)

```
#include <stdio.h>

extern int shared_value; /* DECLARATION: refers to file1.c's variable */

void display(void) {
printf("In file2: shared_value = %d\n", shared_value);
}
```

Output (compile: gcc file1.c file2.c -o app)

```
In file1: shared_value = 100
In file2: shared_value = 150
```

5.6 Advantages and Limitations

Advantages	Limitations
Enables clean sharing of global data across multiple source files.	Overuse creates tight coupling & makes debugging harder (hidden dependencies).
Avoids duplicate memory allocation for the same logical variable.	Reduces modularity/encapsulation if used carelessly.
Supports large modular / team-based project structures.	Risk of multiple-definition linker errors if 'extern' is confused with a definition.

■ **COMMON MISTAKE: extern with Initializer**

Writing **`extern int x = 5;`** actually makes this a **definition** (because it has an initializer), not a pure declaration — a very common exam trick question.

CHAPTER 6

Scope Rules in C

6.1 What is Scope?

Scope defines the region of a program's source text within which a declared identifier (variable/function) can be legally referenced. C recognizes four broad kinds of scope.

6.2 Types of Scope

Scope Type	Description	Example
Block Scope	Identifier declared inside { } is visible only within that block, including nested inner blocks.	{ int x=5; } // x scope ends at }
Local Scope	Same as block/function scope — variables local to a function, invisible outside it.	void f(){ int y; }
File Scope	Identifier declared outside all functions is visible from its declaration point to the end of the file.	int g = 10; // at top of file
Global Scope	File-scope identifiers that are also externally linked — accessible from other files via extern.	extern int g; // in another file

6.3 Diagram: Scope Hierarchy

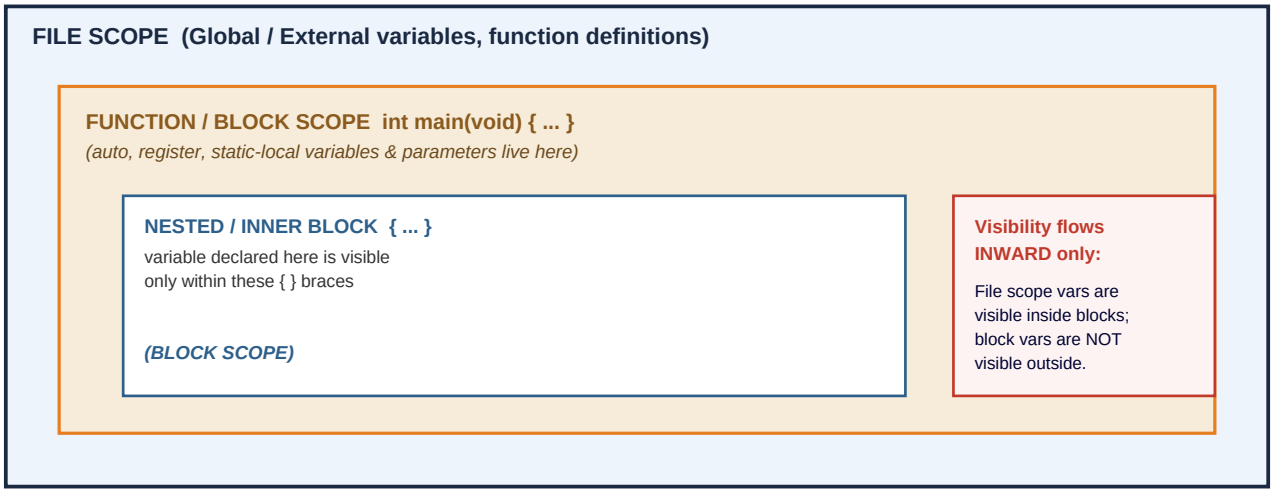

Fig 6.1 — Scope nests from outermost (file) to innermost (nested block); an inner scope can see identifiers from all enclosing outer scopes, but not vice-versa.

6.4 Visibility and Accessibility Rules

- An identifier is visible from its point of **declaration** to the end of the enclosing block/file (not before).
- A local variable with the **same name** as a global variable **hides (shadows)** the global one within its block.
- Variables declared in an **inner block** are not visible in the **outer block**.
- Function parameters have the scope of the function body (essentially block scope of the outermost function block).
- Labels (used with goto) have **function scope** — visible anywhere in the enclosing function.

6.5 Example — Scope and Variable Shadowing

scope_demo.c

```
#include <stdio.h>

int x = 100; /* file/global scope */

int main(void) {
int x = 10; /* local x SHADOWS global x within main */
printf("local x = %d\n", x);
{
int x = 1; /* inner block x shadows main's x here */
printf("inner x = %d\n", x);
}
printf("main x (after block) = %d\n", x);
printf("global x = %d\n", ::x /* NOT valid in C, illustrative only */);
return 0;
}
```

Output (ignore the illustrative last line, which is invalid C syntax)

```
local x = 10
inner x = 1
main x (after block) = 10
```

✓ NOTE: C has no scope-resolution operator

Unlike C++'s `::`, standard C provides **no direct way** to access a shadowed global variable from inside a block where a local variable of the same name exists.

6.6 Local vs Global Variable Diagram

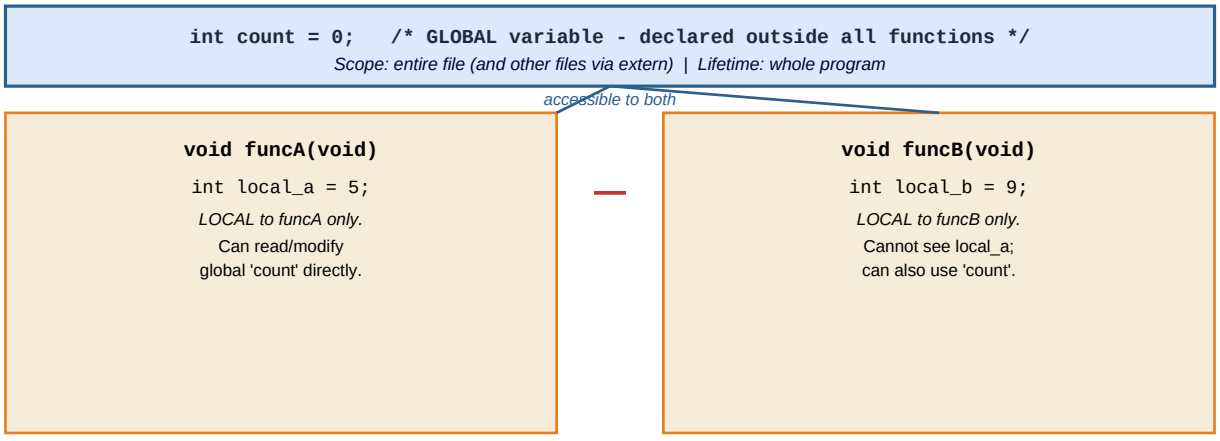

Fig 6.2 — A global variable is visible to every function in the file; local variables are private to their own function.

CHAPTER 7

Comparison of Storage Classes

7.1 auto vs register

Basis	auto	register
Storage location	Stack (RAM)	CPU register (falls back to stack if unavailable)
Speed of access	Normal	Faster (when actually placed in a register)
Address-of (&)	Allowed	NOT allowed
Default keyword?	Yes, default for local vars	Must be stated explicitly
Typical use case	General-purpose local variables	Loop counters / frequently used variables

7.2 static vs extern

Basis	static	extern
Purpose	Retain value (local) / restrict visibility to file (global)	Share a variable across multiple files
Scope	Block (local) or File (global)	Global — spans multiple files
Memory allocated by keyword?	Yes, static allocates storage	No — extern only declares; definition allocates elsewhere
Linkage	Internal linkage (for global static)	External linkage
Visible outside file?	No (even global static is file-private)	Yes (that is its main purpose)

7.3 Scope Comparison (All Storage Classes)

Storage Class	Scope
auto	Block scope (only within its own block/function)
register	Block scope (same as auto)
static (local)	Block scope (only within its own block/function)
static (global)	File scope (entire file, NOT other files)
extern	Global / multi-file scope

7.4 Lifetime Comparison

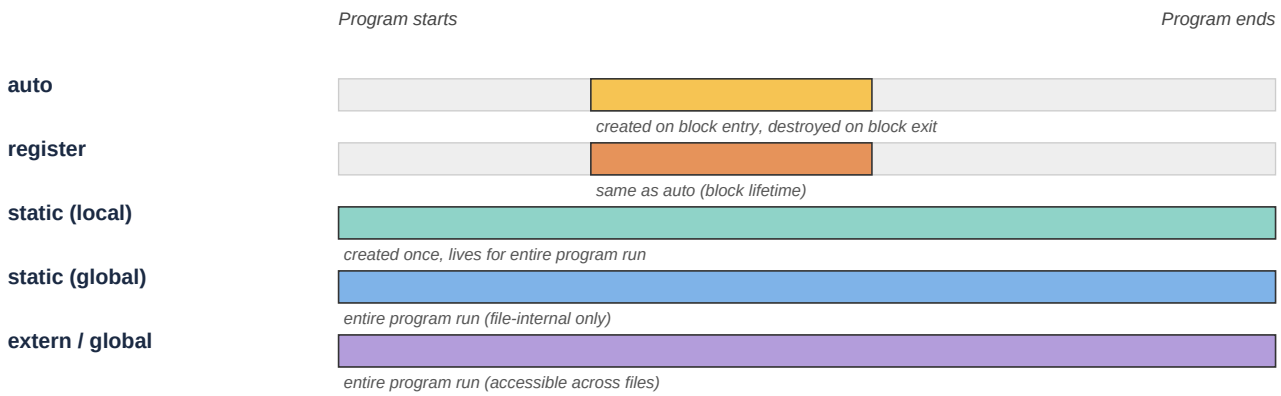

Fig 7.1 — auto/register variables live only for their block's duration; all forms of static and extern variables live for the entire program run.

7.5 Comprehensive Comparison Table (Master Table)

Storage Class	Keyword	Storage Location	Default Value	Scope	Lifetime
Automatic	auto	Stack (RAM)	Garbage	Block	Block execution
Register	register	CPU register / Stack fallback	Garbage	Block	Block execution
Static (local)	static	Data/BSS segment	0	Block	Entire program
Static (global)	static	Data/BSS segment	0	File	Entire program
External	extern	Data/BSS segment	0	Global (multi-file)	Entire program

7.6 Visual Comparison Chart

	auto	register	static (local)	static (global)	extern
Default value	Garbage	Garbage	0	0	0
Memory area	Stack	CPU Reg/Stack	Data/BSS	Data/BSS	Data/BSS
Lifetime	Block	Block	Program	Program	Program
Scope	Block	Block	Block/File	File	Multi-file

CHAPTER 8

Programming Examples

8.1 Program Demonstrating All Four Storage Classes Together

all_storage_classes.c

```
#include <stdio.h>

int globalVar = 5; /* extern by default — visible to other files too */
static int fileVar = 10; /* static global — visible only in this file */

void demo(void) {
auto int a = 1; /* explicit auto (same as just 'int a = 1;') */
register int r = 2; /* hint: keep r in a CPU register */
static int s = 0; /* retains value between calls */

a++; r++; s++;
printf("a=%d r=%d s=%d global=%d fileVar=%d\n",
a, r, s, globalVar, fileVar);
}

int main(void) {
demo();
demo();
demo();
return 0;
}
```

Output

```
a=2 r=3 s=1 global=5 fileVar=10
a=2 r=3 s=2 global=5 fileVar=10
a=2 r=3 s=3 global=5 fileVar=10
```

Observe: `a` and `r` reset every call (auto/register behaviour), while `s` keeps incrementing (static behaviour).

8.2 Scope-Based Example — Nested Blocks

nested_scope.c

```
#include <stdio.h>
int main(void) {
int level = 1;
printf("Outer level = %d\n", level);
{
int level = 2; /* shadows outer 'level' */
printf("Middle level = %d\n", level);
{
```

```
int level = 3; /* shadows middle 'level' */
printf("Inner level = %d\n", level);
}
printf("Back to middle level = %d\n", level);
}
printf("Back to outer level = %d\n", level);
return 0;
}
```

Output

```
Outer level = 1
Middle level = 2
Inner level = 3
Back to middle level = 2
Back to outer level = 1
```

8.3 Multiple-File Example Using extern (Recap)

See Section 5.5 for a complete two-file **`extern`** example (`file1.c` defines `shared_value`; `file2.c` declares it with **`extern`** and uses it). Compile both files together: **`gcc file1.c file2.c -o app`**.

8.4 Common Mistakes and Debugging Tips

Mistake	Why It Fails / Fix
Taking &register_var	Compile error — register variables may not have a memory address. Remove 'register' or don't take its address.
Expecting auto var to retain value	auto variables reset every call; use 'static' if persistence is required.
extern int x = 5; used as declaration	This is actually a DEFINITION (has initializer) — duplicate definitions across files cause linker errors.
Defining the same global variable in two files	'Multiple definition' linker error — define once, declare with extern elsewhere.
Assuming static local vars start fresh	static locals are initialized only ONCE; the initializer is skipped on later calls.
Forgetting static globals are file-private	extern cannot expose a 'static' global to another file — by design.

CHAPTER 9

Quick Revision Section

9.1 Important Definitions (One-Liners)

Storage Class: Specifies scope, lifetime, storage location, and default value of a variable.

auto: Default storage class for local variables; block scope, block lifetime, garbage default value.

register: Requests CPU-register storage for faster access; no address-of allowed; falls back to stack.

static (local): Local variable that retains its value between function calls; initialized once; default 0.

static (global): Global variable restricted to file scope only; not accessible via extern in other files.

extern: Declares a variable/function defined elsewhere, enabling sharing across multiple files.

Scope: The region of code where an identifier's name is valid/visible.

Lifetime / Duration: The span of program execution during which a variable exists in memory.

9.2 Scope & Lifetime Summary Table

Storage Class	Scope	Lifetime	Default Value
auto	Block	Block execution only	Garbage
register	Block	Block execution only	Garbage
static (local)	Block	Entire program	0
static (global)	File	Entire program	0
extern	Global (multi-file)	Entire program	0

9.3 Frequently Asked Exam Questions

Q1. What is the default storage class of a local variable in C?

auto — if no storage class is specified for a local variable, it is automatically 'auto'.

Q2. Why can't we use & (address-of) with a register variable?

Because the compiler may store it in a CPU register, which has no memory address; C forbids taking its address even if it falls back to RAM.

Q3. What is the default value of an uninitialized static variable?

Zero (0) — static and extern variables are automatically zero-initialized if not given an explicit value.

Q4. Differentiate between a local static variable and a global static variable.

A local static variable has block scope but program lifetime and retains value between calls; a global static variable has file scope, program lifetime, and is hidden from other files.

Q5. What is the purpose of the extern keyword?

To declare a variable or function that is defined in another file (or later in the same file), enabling sharing of that identifier across multiple source files.

Q6. Can a static global variable be accessed in another file using extern?

No. 'static' at file scope gives internal linkage, restricting the variable strictly to its own file, regardless of extern declarations elsewhere.

Q7. What happens if you write 'extern int x = 5;'?

It becomes a definition (not a pure declaration) because it has an initializer; this can cause a 'multiple definition' error if x is also defined elsewhere.

Q8. Which storage classes have block scope and which have file/global scope?

Block scope: auto, register, static(local). File scope: static(global). Global/multi-file scope: extern.

Q9. What is variable shadowing?

When an inner-block variable has the same name as an outer-block/global variable, the inner one temporarily hides (shadows) the outer one within its own block.

Q10. Which storage classes are stored in the Data/BSS segment rather than the stack?

static (local & global) and extern variables; auto and register variables are stored in the stack (or CPU register).

9.4 One-Page Revision Notes (Final Recap)

- **auto** — default for locals; stack; block scope & lifetime; garbage default.
- **register** — CPU register hint; no &address; block scope & lifetime; garbage default.
- **static (local)** — retains value across calls; initialized once; block scope, program lifetime; default 0.
- **static (global)** — file-only visibility; program lifetime; default 0; blocks external linkage.
- **extern** — declares (not defines); shares variables across files; global scope; program lifetime; default 0.
- **Scope order (narrowest→widest):** nested block < block/function < file < multi-file (extern).
- **Storage location split:** Stack/Register → auto, register. Data/BSS → static, extern.
- **Only static & extern variables default-initialize to 0**; auto & register default to garbage.

End of Study Guide — Storage Classes in C. Good luck with your examination!